The aim is to build a full-duplex, voice-enabled AI agent running on a Raspberry Pi (4 or 5). Your responsibilities:

- **Install AI agent harness (e.g. onit, pipecat, crew)** with MCP support on RPi
- Build MCP servers that to SOTA ASR and TTS
- Install VAD on RPi for voice detection,
- Run a full-duplex AI agent that can real-world tasks
- You may use commercial APIs for models but you build and run your MCP servers (on RPi)
- Build a latency benchmark (classwork): a standardized latency measurement method. Note that this is . Speech interaction is natural, conversational and interruptible.

—----------

What we tried

- **Pipecat** - end to end solution
    - Out of the box, nagagawa na yung basic prompts
    - Pag agentic tasks -> medyo hindi ok
    - Set a timer, tool calling not sure pa
- **Crew** - enterprise $$$$
- ASR/TTS - deepgram, cartesia, MLX, Whisper
- Moshiko/Personaplex (voicechat) by NVIDIA - speech-to-specech 4b -> I think we avoid speech-to-speech models but I think pwede naman? Tried this with hermes hindi niya kaya mag-delegate ng totoong task sa -Jus

—----------

What we need

A. **Full Duplex Pipeline (basically it should work like GPT-LIVE-1)**
    a. The agent should live in the RPi, but it should avoid model inference.
    b. Memory (or we use the harness memory)
    c. Wake word - VAD is required but wake word is optional
        i. Strategy is to ensure only your voice is detected (useful for demo)
    d. The full duplex agent should be able to delegate hard tasks to an available harness or quickly make a tool call for “easy” tasks.
    e. Connectable to already available agentic frameworks like Onit, Hermes,OpenCode, pi.dev via MCP
        i. The harness is responsible for working on “complex tasks” like Creating a Website, giving a report, etc.
        ii. Nakakausap pa rin ng person yung agent while the harness is working in the background
        iii. Get back on the results
    f. Acknowledgement replies - backchanneling
        i. “Mhm?”
        ii. “Yes?”
        iii. “Let me check that” -> Delegate task to harness

B. **Benchmarking** - generate a single bank of commands for benchmarking. natural, conversational and interruptible.
    a. Real-time responses
    b. Turnaround time
        i. Between agent harness turns (bare minimum: treat the harness as a blackbox)
    c. LLM as a Judge?
    d. How natural a conversation feels
        i. Backchanneling metrics
        ii. Pause detection
        iii. Interruption -> is it backchannel or genuinely steering?
    e. V1.5 paper (King):
        i. How much it tries to respond? Annoying ba siya?
        ii. Overlapping events -> we need to know whether backchannel lang siya or legit na interruption. Model can pause tas tuloy yung sinasabi niya OR ignore it
        iii. Count kailan nag-silence once naka-detect ng overlapping event at gaano kabilis siya mag-respond based sa bagong command
        iv. Mesure if the tone is consistent with the before tone
    f. **Unified benchmarking:** have a bank of commands, only sample some for each trial, we can read out the script on the pi for live testing. We copy the method of lin et al 2026 for testing (inserting pauses, backchanneling, interruptions, etc)
    g. Final Set of variables:
        i. <insert here sa 231 meeting>

—---------

Action Items

- Have a standard codebase for the benchmark? *Thinking of having 2-3 students maintain the repository but we should still have write access to them.*
- Free up time after AI 231 class.
- Consult with Sir Rowel after?
    - Collate as many questions as possible
- Read and understand the FullDuplexBench articles
    - https://arxiv.org/abs/2503.04721
    - https://github.com/DanielLin94144/Full-Duplex-Bench
    - V1.5 benchmark (see King's message)

Open Questions to ask Sir Rowel

- **Can we allocate an A100 as 24/7 inference so we don't have to pay for ASR/TTS? How about a lightweight LLM so we don't clog the QWEN 3.8 for simple testing.**

Zoom Meeting Recording Link:

Shareable link:
https://up-edu.zoom.us/rec/share/AcU_-1cZEUrUFpRxvNWXEnUz5ZrT8h2O42n4Z2Hwshux_2t1Fy6aKITfQou3zdlz.SOOuOXxlgClqCKym
Passcode: ZB7mtou#

Agent capabilities to scope:

- Planning and multi-step execution: Agents plan, execute multi-step tasks, correct course, and hand off to other systems.
- Tool use: This covers retrieval, function and API calling, code execution, and browser or computer use. Coding agents can read repos, edit files, and run tests.
- Computer and browser use: Models look at screens, click, and operate GUIs, so the computer interface itself becomes a tool.
- Long-running work: Systems are starting to work for hours or days, coordinate with other agents, and use real tools.
- Multi-agent teams: Multi-agent systems are becoming a default architecture, because single agents struggle with long, multi-step processes.
- Memory and skills: Agents are gaining long-term memory (preferences, project rules, history) and tool networks via MCP and Skills.
- Enterprise integration: Agents can trigger workflows, call APIs, and move data between systems.